MIT 8103 Advanced Database Systems: Continuous Assessment Final Documentation

Nov 12, 2026 · Aregbe Abiodun Micheal

All five portfolio activities were completed on one case study, LagosLink Deliveries. The scripts, screenshots and READMEs are in the GitHub repository.

Student: Aregbe Abiodun Micheal (Student ID 301915034, matric 2026/A/MIT/0502)

Course: MIT 8103 Advanced Database Systems, Miva Open University, Continuous Assessment, 2026/2027 first semester

GitHub: github.com/AregbeAMicheal/advanced-database-portfolio

Tools: PostgreSQL 18, pgAdmin 4, MongoDB Atlas (free M0, AWS Paris), MongoDB Compass

Evidence: each section links to its GitHub folder. Key screenshots are in the accompanying file **Aregbe_Abiodun_301915034_Evidence.pdf**: Portfolio 1 Figures 1–3, Portfolio 2 Figures 4–5, Portfolio 3 Figures 6–7, Portfolio 4 Figures 8–9 and Portfolio 5 Figures 10–12.

1. Case study

LagosLink Deliveries is a simulation of a parcel delivery company in Lagos, based on the Ride-hailing / Logistics System choice in the brief. The same company is used in all five portfolios and all names, phone numbers and addresses are made up.

- **Nature of the business:** same-day delivery of parcels from three hubs (Ikeja, Yaba and Lekki) using motorbike and van drivers.
- **Users of the database:** customers (both businesses and individuals) who order deliveries, dispatchers who assign drivers to them, drivers who report the delivery stages, hub managers, and finance department staff.
- **Main data:** hubs, customers, drivers, vehicles, deliveries, delivery status updates, payments and GPS positions of drivers.
- **Main transactions:** order a delivery, assign the nearest available driver, record delivery stage changes, record payments, track a delivery and produce reports on revenue and on-time performance per hub.

2. Portfolio 1: Database design and modelling

I created an ER diagram (pgAdmin ERD Tool) and 8 tables in third normal form: hubs, customers, drivers, vehicles, deliveries, delivery_status_updates, payments and driver_locations. These tables have 8 primary keys, 8 foreign keys, 7 unique and 14 check constraints. 8 test transactions designed to break some constraint (e.g., a duplicate key, a customer does not exist, 'status = Lost', a delivery deadline earlier than the date of booking, etc.) have been refused. There is one exception of a deliberately denormalised column deliveries.status needed to speed up parcel tracking. Evidence

3. Portfolio 2: Query processing and optimisation

With 50,012 deliveries, 235,212 delivery status updates and 200,007 GPS points for drivers, **EXPLAIN ANALYZE** showed sequential scans for three common queries. After creating 3 composite indexes, they became index scans. Evidence

Query	Before	After	Speed-up
Q1 A parcel's status history	76.484 ms	0.232 ms	330×
Q2 A customer's deliveries, newest first	3.461 ms	0.223 ms	15×
Q3 A driver's latest GPS point	63.151 ms	0.031 ms	2,037×

The greatest gains were achieved in situations where a query needs only a few rows from a big table. The cost is about 15 MB of indexes and slightly decreased insert performance.

4. Portfolio 3: Transactions and concurrency

Two connections of pgAdmin represented two dispatchers. First, I demonstrated COMMIT, ROLLBACK and atomicity: an error in a CHECK constraint cancelled the entire three-step transaction. Next, I showed the absence of dirty reads under READ COMMITTED, and stability of a snapshot under REPEATABLE READ (₦5,000 is still visible after ₦4,500 is committed). Then, I demonstrated a lost update, where driver 6 silently overwrote driver 5. With SELECT ... FOR UPDATE, the second dispatcher waited about 62 seconds, detected that the job was taken and ROLLBACKED. Evidence

5. Portfolio 4: NoSQL and advanced data models

In MongoDB Atlas, each delivery is represented by one document with its customer, hub, status history and payments embedded. Drivers are referenced by driver_id as they change frequently. The collections have 3 hubs, 9 drivers and 2,012 deliveries.

- Tracking a delivery required one read without a join (Q1), and marking it as delivered one atomic update (Q2).
- Aggregation pipeline generated a hub report, e.g., for Ikeja: 599 deliveries, ₦2,295,000, 61.3% on time (Q3).
- $geoNear found the nearest free driver, 1.86 km away and excluding busy drivers (Q4). $lookup joined driver's information (Q5).
- Schema validator rejected status 'Lost' (Q6), and an index reduced number of documents to read from 2,012 to 5 (Q7).

The trade-off is duplicate customer information and no enforced foreign keys. Evidence

6. Portfolio 5: Distributed and cloud databases

The data runs on an Atlas **replica set of 3 nodes** (AWS Paris), secured by a database user, an IP access list and TLS. Evidence

- **Replication:** db.hello() showed set atlas-wm9buy-shard-0, with shard-00-01 as the primary.
- **Read preference:** the same read was answered by the primary by default, and by secondary shard-00-02 with readPref('secondary').
- **Write concern:** 20 writes averaged 152 ms with both w: 1 and w: "majority". The Lagos to Paris trip dominates, so payments can use the safer setting at no visible cost.
- **Read concern:** a payment saved with w: "majority" was read back with read concern "majority".
- **Designed, not run:** a failover test and sharding by hub; both need paid tiers.

7. Individual technical reflection

The most valuable lesson was how one business, LagosLink Deliveries, behaved under five different database ideas. The normalised PostgreSQL design protected data quality, and all eight constraint tests were refused as intended. Indexing showed the clearest payoff: the latest-GPS-point query fell from 63.151 ms to 0.031 ms. The concurrency work was the most instructive, because a lost update raised no error at all; only SELECT … FOR UPDATE stopped driver 6 from overwriting driver 5. In MongoDB, embedding the status history made tracking a single read, while drivers stayed referenced because they change constantly. Several problems shaped the process: my first ER diagram had three relationships drawn the wrong way, which i spotted; whole scripts were sometimes run instead of single tests; and Compass failed until a changed IP address was added to Atlas. The final surprise was that majority write concern cost nothing extra from Lagos: both settings averaged 152 ms.

8. References

1. Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
2. Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Harlow: Pearson.
3. Gilbert, S. and Lynch, N. (2002) 'Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services', *ACM SIGACT News*, 33(2), pp. 51–59.
4. PostgreSQL Global Development Group (2025) *PostgreSQL 18 Documentation*: Constraints, Using EXPLAIN, Transaction Isolation, Explicit Locking.
5. pgAdmin Development Team (n.d.) *pgAdmin 4 Documentation* (Query Tool, ERD Tool). Available at: https://www.pgadmin.org/docs/pgadmin4/latest/
6. MongoDB, Inc. (n.d.) *MongoDB Manual*: Data Modeling, Aggregation, Replication, Write Concern, Read Concern, Sharding.
7. MongoDB, Inc. (n.d.) *MongoDB Atlas Documentation* and *MongoDB Compass Documentation*. Available at: https://www.mongodb.com/docs/atlas/ and https://www.mongodb.com/docs/compass/. Online sources accessed November 2026.

Data and libraries: all data is simulated and was generated by my own scripts (PostgreSQL generate_series, and a fixed-seed generator in MongoDB). No external datasets, code libraries or real company data were used.